Middlesex University London

MODULE HANDBOOK

CST1510

Programming for Data Communication and Networks

Faculty of Science and Technology

Department of Computer Science

Academic Year 2026-27

Semester 1

Middlesex University London

Contents

Welcome to Middlesex! 3

Your Module Teaching Team 3

Module Overview 3

Assessment 9

Assessment brief 10

Academic Integrity and Misconduct 12

Support that is available to you 13

Feedback on your assignments 15

Results Confirmation 15

Expectations of studying 16

Appendix 1: Marking Criteria 17

If you have a disability which makes navigating this handbook difficult and you would like to receive information in an alternative format, please contact campus life.

Welcome to Middlesex!

Welcome to CST1510 Programming for Data Communication and Networks. This module introduces you to computer programming using Python, one of the most widely used languages in industry and research, valued for its readability and its central role in data science, automation and software development. No prior programming experience is assumed or required.

Across twelve weeks, you will build your programming ability incrementally: starting with the fundamentals of values, control flow and functions, progressing through core data structures and file handling, and culminating in a continuous project in which you pull in real external data, secure it, store it properly, and deploy it as a working web application. Each week follows a consistent rhythm of a workshop and a lab, so that new concepts are introduced and then immediately practised.

Your learning is assessed through two courseworks: CW1, a skills portfolio covering Weeks 1–7, and CW2, a continuous project developed across Weeks 8–12. Together, these give you the opportunity to demonstrate both your grasp of individual programming techniques and your ability to combine them into a complete, working piece of software.

The purpose of this Handbook is to introduce you to your module, learning goals and to direct you to other general information about studying at Middlesex University. The material in this document is as accurate as possible at the date of production; however, you will be informed of any major changes in a timely manner

This handbook can also be accessed via My Learning at: link

Your Module Teaching Team

Staff Name	Contact Details	Role Description
Anna Wieclaw	n/a	Module Leader - Hendon
Aisha Idoo	a.idoo@mdx.ac.mu	Module Coordinator
Haardik Gautam	h.gautam@mdx.ac.mu	Lab tutor

Module Overview

Aims

The main aim is to provide you with the opportunity to learn programming effectively. This takes place in the context of learning to solve a range of problems arising in the context of the processing, storing and transmitting data. You will understand the role of a basic but important range of data structures and types, and their use. Several data structures of interest within the networking and security fields will be considered, for example those to represent RTP packets (and associated operations on these) in order that audio- visual data can be sent using transport protocols such as the UDP protocol. The concept of an algorithm will be introduced early on together with the techniques for program implementation. You

will be encouraged and supported in undertaking - both individually and within a small group setting - a variety of problemsolving tasks.

Knowledge

On successful completion of this module, the student will be able to:

1. Articulate the use and meaning of a range of key programming constructs.

2. Select (and design) data structures for given programming purposes.

3. Demonstrate and explain basic algorithms and the use of operations on a variety of data structures; searching and sorting of a list, and basic techniques for transferring data between computer components, and between networked computers.

Skills

On successful completion of this module, the student will be able to:

4. Plan, develop, test and debug programs to solve a range of problems using recognised principles.

5. Explain the operation of a selection of given programs.

6. Produce code to a professional standard, including comments, indentation and other meta-information.

7. Operate effectively to carry out collective programming tasks both individually and as a group member.

Syllabus

Basic terminology: compilers, interpreters, debugging and testing software, variables, expressions and statements data structures and types, operations, functional, functions; function calls, composition; flow of control/execution; parameters and arguments; stacks, frames and return values; program documentation. Conditionals and recursive definition. Integers, real numbers, mathematical and logical operators; Boolean expressions; nested conditionals; device, machine and network I/O and operations. Incremental program development; type checking; destructive and nondestructive assignment; iteration; while loops; break statement; algorithms; strings; for-loops; searching and sorting; global variables. Objected oriented languages, their main features and approach. Objectoriented approach to programming: classes, methods, inheritance and object creation; files; paths; classes and types; instances as return types; the role and use of prototyping. Introduction to sockets and streams

Planner

Week 1 - Values, Types and Input/Output

Topics Covered: Variables and data types (int, float, str, bool), type conversion, arithmetic, user input with input(), and formatted output with f-strings.

What You'll Learn: This week lays the Python foundations — storing values in well-named variables, converting between types, and turning user input into a tidy, formatted report. You'll also learn to read and diagnose basic error messages.

Week 2 - Decisions and Loops

Topics Covered: Comparisons and boolean logic (and, or, not), conditional statements (if/elif/else), while loops, for loops with range(), and enumerate().

What You'll Learn: You'll learn to make programs branch and repeat — writing code that takes different paths depending on a condition, and loops that repeat a known or unknown number of times safely.

Week 3 - Functions and Modules

Topics Covered: Defining and calling functions, parameters, default and keyword arguments, *args/**kwargs, variable scope, return vs print, docstrings, importing modules, and if __name__ == "__main__".

What You'll Learn: You'll learn to break code into reusable, named pieces — functions that take inputs, return values properly, and can be imported and reused across files.

Week 4 - Lists, Dictionaries and Files

Topics Covered: Lists (indexing, slicing, comprehensions), tuples, dictionaries, sets, reading/writing files with with, and handling errors with try/except.

What You'll Learn: You'll learn Python's core data structures for storing collections of data, how to persist it to files safely, and how to catch specific errors — building up to the shape most real data comes in: a list of dictionaries loaded from a file.

Week 5 - Classes and Objects

Topics Covered: Defining classes with __init__, the self parameter, attributes and dot notation, methods, and working with lists of objects.

What You'll Learn: You'll learn to bundle related data and behaviour into objects — the core idea of object-oriented programming — and see how a method can change its own object's state.

Week 6 - Pandas

Topics Covered: Series and DataFrames, pd.read_csv(), selecting/filtering with loc/iloc, groupby() and aggregation, sorting, and lambda functions with .apply().

What You'll Learn: You'll learn to work with real tabular data efficiently using pandas — loading, filtering, summarising and charting a dataset in a fraction of the code a manual loop would take.

Week 7 - Strings and Text Manipulation

Topics Covered: Case and whitespace methods, searching and replacing text, splitting and joining strings, and parsing messy real-world text into clean data.

What You'll Learn: You'll deepen your string-handling skills to clean and parse real, messy text — the last toolkit-building week before the CW2 project begins.

Week 8 - Pulling Real Data

Topics Covered: Calling web APIs with requests, handling failed/slow requests, downloading files from the web, regex with re.search()/re.findall()/re.sub(), and refactoring functions into a class.

What You'll Learn: This is the first week of CW2. You'll pull real data from outside your program — a live API and files from the web — and start shaping your code into a class, the structure your CW2 project grows from.

Week 9 - Hashing and Auth

Topics Covered: Why passwords are hashed (not encrypted), salting and rainbow table attacks, hashing/verifying passwords with bcrypt, and building a safe register/login flow.

What You'll Learn: You'll learn to handle user credentials safely — hashing and verifying passwords with bcrypt, building a real login gate, and keeping secrets out of your code and repository.

Week 10 - SQL, Properly

Topics Covered: Connecting to SQLite, creating tables, safely inserting/querying/updating/deleting rows with placeholders, SQL injection, and migrating flat-file data into a database.

What You'll Learn: You'll replace flat files with a real database — writing safe SQL, understanding SQL injection and how parameterised queries prevent it, and migrating your project's data into proper tables.

Week 11 - Streamlit Dashboard

Topics Covered: Streamlit's rerun model, st.dataframe()/st.metric(), interactive widgets, st.session_state, wiring in login, layout (columns/sidebar/tabs), caching, and deployment.

What You'll Learn: You'll turn your project into a real, interactive web app — displaying your data, protecting pages behind login, and deploying it to a public URL.

Week 12 - Intelligence (Optional) and Deployment

Topics Covered: Safely loading API keys, calling the ChatGPT API, writing pytest tests (including mocking external calls), a correct requirements.txt/.gitignore, and deployment.

What You'll Learn: In the final week, you'll optionally add an AI feature to your project, learn to test code that calls external services without actually calling them, and deploy your finished app to a public URL.

Week Beginning	Week	Lab	Summative Assessment
Sep 23rd	1	Values, Types and Input/Output. Variables and data types, type conversion, arithmetic. User input and f-string output.	
Sep 30th	2	Decisions and Loops. Comparisons and boolean logic. if/elif/else, while loops, for loops with range(), enumerate().	

Week Beginning	Week	Lab	Summative Assessment
Oct 7th	3	Functions and Modules. Parameters, default/keyword arguments, *args/**kwargs. Scope, return vs print, importing modules.	
Oct 14th	4	Lists, Dictionaries and Files. Lists, tuples, dictionaries, sets. Reading/writing files, try/except.	
Oct 21st	5	Classes and Objects. __init__, self, attributes and methods. Working with lists of objects.	
Oct 28th	6	Enrichment. Guest workshop: Git.	
Nov 4th	7	Pandas. Series and DataFrames, read_csv(), loc/iloc. groupby(), sorting, lambda with .apply().	CW1 – Skills Portfolio submission (50%). Tiered mini-projects and handwritten Cheat Sheet, Weeks 1–7. ***Due end of Week 7.***
Nov 11th	8	Strings and Text Manipulation. Case and whitespace methods, search and replace. Splitting, joining and parsing messy text.	
Nov 18th	9	Pulling Real Data. Web APIs with requests, handling failed requests. Regex, refactoring functions into a class.	
Nov 25th	10	Hashing and Auth. Hashing vs encryption, salting. bcrypt, register/login flow.	

Week Beginning	Week	Lab	Summative Assessment
Dec 2nd	11	SQL, Properly. SQLite, creating tables, CRUD with placeholders. SQL injection, migrating flat files to a database.	
Dec 9th	12	Streamlit Dashboard. Rerun model, widgets, session_state. Login, layout, caching, deployment.	
Dec 16th	13	Intelligence (Optional) and Deployment. API keys, ChatGPT API. pytest and mocking, requirements.txt/.gitignore, deployment.	
Dec 23rd			CW2 – Continuous Project submission (50%). Deployed Streamlit application. ***Coursework 2 Due***

Assessment scheme

The module is assessed through two pieces of coursework, each contributing 50% of the module mark. CW1 is a skills portfolio, submitted at the end of Week 7, which evidences your growing command of core Python programming techniques covered from Week 1 onwards. CW2 is a continuous project, started in Week 8 and submitted one week after the end of Week 12, in which you apply and extend those skills to build, secure and deploy a working piece of software using real external data. This module has no examination.

Scheduled teaching: 60 hours;

Independent study: 228 hours

Learning Resources

Essential M. Lutz, *Learning Python*, 6th ed. Sebastopol, CA: O'Reilly Media.

Recommended C. Althoff, *The Self-Taught Programmer: The Definitive Guide to Programming Professionally*. Self-published.

Official documentation Python Software Foundation, *The Python Standard Library* and *The Python Tutorial*, available at: https://docs.python.org/3/

Official documentation for the third-party libraries used across the module — including pandas, Requests, bcrypt, sqlite3 and Streamlit — is linked from the relevant week's materials on My Learning, and should be treated as a primary reference alongside the essential text.

Module materials Weekly workshop and lab materials, walkthrough notebooks, demonstrations and lab briefs are provided on My Learning for each week of the module and should be worked through in sequence.

Assessment

Formative assessment: These activities help monitor your learning and provide feedback, but do not contribute to your final module mark.

Formative assessment	Deadline
Weekly lab tasks, completed during the lab session (Weeks 1–12)	End of each lab session
GitHub commit and push of the week's lab work — your weekly evidence of progress	Same day as the lab session

Summative assessment: This type of assessment evaluates your overall learning for the module and determines your final grade. The table below outlines the associated deadlines.

Summative assessment	Weighting	Deadline	Feedback
CW1 – Skills Portfolio: weekly tiered mini-projects (Threshold/Typical/Excellent) and handwritten Cheat Sheet, Weeks 1–7	50%	End of Week 7	15 working days after submission
CW2 – Continuous Project: one project grown weekly in your chosen lane (AI/Data Science, Cyber Security, or IT), ending in a deployed Streamlit application, Weeks 8–12	50%	One week after the end of Week 12	15 working days after submission

To complete this module, you need to pass all assessments with a minimum grade of 16 or equivalent, with a minimum grade of 18 in each individual assessment component. No additional Professional, Regulatory or Statutory Body (PRSB) requirements apply to this module.

Before submitting your work, ensure all sources are accurately referenced and that spelling, punctuation, and grammar are correct. Written assessments evaluate technical proficiency in English; details are outlined in the assessment marking criteria and the University Grade Criteria Guide.

Reasonable adjustments will be made for those students who have a declared disability/specific learning condition which would affect performance in this area.

Reassessment for this module normally takes place in the following way:

Reassessment normally takes the form of a repeat submission of the failed coursework component(s) CW1 and/or CW2 during the University's designated resit period. As this module is assessed entirely through coursework, with no examination component, there are no PSRB requirements constraining reassessment.

Assessment brief — CW1

Field	Detail
Assessment	CW1 — Skills Portfolio
Assessment Type	Portfolio (individual)
Description	A weekly skills portfolio covering Weeks 1–7. Each week you complete the lab notebook (warm-up plus all seven drills), a mini-project at your chosen tier (Threshold, Typical or Excellent), and a handwritten journal page. At the end of Week 7 you submit a completed Cheat Sheet (Boxes 1–7).
Submission date, time	End of Week 7, at the submission time published by the module team
Feedback type & date	Written feedback, within 15 working days of submission
Word count (or equivalent)	N/A — evidenced through notebooks, code, journal pages and Cheat Sheet
Assessed learning outcome(s) & evidence	Explains core Python principles (data types, control structures, functions, OOP) and applies them to build working programs; evidenced through weekly lab notebooks, mini-projects and journal entries
Assessment weighting %	50%

Field	Detail
Submission Method	Submit the GitHub repository link through Moodle
Assessment Criteria	Python knowledge and application (35%), working mini-projects (25%), problem solving and debugging (15%), code quality and good practice (15%), learning evidence and portfolio completion (10%)
Use of GenAI Tools	RED — no GenAI use for the handwritten journal pages or Cheat Sheet; these must be your own, handwritten work. AMBER — assistive use only for the weekly lab notebooks and mini-projects (e.g. IDE-integrated tools for debugging), once you have worked through the fundamentals yourself. GenAI must not produce code you cannot explain.

Assessment brief — CW2

Field	Detail
Assessment	CW2 — Continuous Project
Assessment Type	Individual project, with demo video and technical report
Description	One continuous application, built in your chosen lane (AI/Data Science, Cyber Security, or IT), grown week by week across Weeks 8–12: external input and classes, authentication, SQLite storage, a Streamlit dashboard, and deployment. Submission includes the final folder, five journal pages, Cheat Sheet Boxes 8–12, a demo video, a code walkthrough, a technical report, and the deployed application's public URL.
Submission date, time	One week after the end of Week 12, at the submission time published by the module team
Feedback type & date	Written feedback, within 15 working days of submission
Word count (or equivalent)	Technical report + demo video and code walkthrough (word count/duration to be confirmed in the CW2 brief)
Assessed learning outcome(s) & evidence	Handles data storage, retrieval and processing via files, databases and web APIs; identifies and mitigates security vulnerabilities (insecure password storage, SQL injection); builds, tests and deploys a working Streamlit application with secure authentication — evidenced through the deployed project, its commit history, and the final presentation evidence
Assessment weighting %	50%
Submission Method	Submit the GitHub repository link, deployed application URL, demo video, code walkthrough, and technical report through Moodle
Assessment Criteria	Functional requirements and progression (25%), technical implementation (20%), security/reliability/data handling (20%), testing and quality assurance (15%), user experience and deployment (10%), development process and technical communication (10%)
Use of GenAI Tools	RED — no GenAI use for the handwritten journal pages or Cheat Sheet. AMBER — assistive use only for project code (e.g. debugging support, IDE-integrated tools); you must be able to explain your code, development decisions and solution in detail, as demonstrated in the required video walkthrough.

For assessment criteria, please refer to Appendix 1 at the end of this document.

Academic Integrity and Misconduct

Academic integrity means producing your work in a transparent way, based on valid academic and professional sources, properly referenced and with any assistance fully acknowledged. Refer to the 'Academic Integrity' guidance of MyMdx for further details.

Academic misconduct takes several forms, in particular:

- **Plagiarism** – using extensive unacknowledged quotations from, or direct copying of, another person's work and presenting it for assessment as if it were your own effort.
- **Purchasing or Commissioning -** This includes the use of third party essay writing services. Essay mills are now illegal entities and use of them is facilitating an illegal activity.
- **Collusion** – working together with other students (without the tutor's permission) and presenting similar or identical work for assessment.
- **[If applicable] Infringement of Exam Room Rules** – Communication with another candidate, taking notes to your table in the exam room and/or referring to notes during the examination.
- **Self-Plagiarism** – including any material which is identical or substantially similar to material that has already been submitted by you for another assessment in the University or elsewhere.
- **Unauthorised use of Generative Artificial Intelligence (GenAI)** – using GenAI without appropriate acknowledgement and referencing. Appropriate use of GenAI is detailed in the assessment requirements grid in **section 7.5** and abides by our **University's GenAI Policy**.

To support your academic integrity, please refer to the Referencing guidance and ensure your referencing meets your discipline's required standards.

Appropriate use of GenAI

This table defines the different levels of scope you may be given to use GenAI in your assessed work. Your Assessment Brief above defines which level applies to that assessment task and will include further guidance and support.

GenAI Scope in Assessment	What it means
RED – No scope	GenAI is not permitted in the development or creation of outputs (except assistive technology via reasonable adjustment). Assessments in this category can include exams, Vivas/interviews, and professional / regulatory competencies where GenAI is out of scope.
AMBER – Assistive scope	Assistive use is permitted only within defined parameters for assessment preparation and process, such as: summarising sources, connecting ideas/themes, self-

	test and revision prompts, spelling and grammar support in drafts, support for data analysis, project planning and management. GenAI should not be used to produce entire sections of text or elements of media that appear in the final submission.
GREEN– Integrated Scope	Integrated use: GenAI can be used to create content that appears in the final submission. GenAI can be used at multiple stages of the assessment process. GenAI use must be acknowledged and/or cited, and assessments may ask for student commentary and critical evaluation of its use.

Record and Acknowledgement of GenAI Use

Use this table to record how you have used GenAI for this assessment. It will help you keep track of the tools and prompts you used, reflect on your learning, and explain your study choices if needed - for example in tutorials, group work, reflections, or reflective and methodological sections of your work.

GenAI tool used	Date of use	Amber or Green Category & Purpose of use	Examples of prompts used	How did you check the validity of the Gen-AI suggestions?	The section or phase of assessment where GenAI was used
e.g. *MS Copilot, Chat GPT, Midjourney, Perplexity etc*	*e.g.15/07/2026*	*e.g.* Amber, spell checking; *Green, 'To suggest an outline essay structure'*	e.g. 'Generate ideas for structuring an essay on the themes of…'	e.g. Cross checked against reading list materials; Google scholar; Discussion with tutor	e.g. Planning project work Finding themes in data Producing an outline structure Checking draft sections for grammar Integrating and evaluating GenAI output in final work

Support that is available to you

Support and draft feedback sessions for *Insert assessment*

Coursework briefing

CW1 -

Each week's mini-project is briefed at the end of that week's 2-hour workshop, before students move into the 3-hour lab. The overall CW1 portfolio requirement — including the Cheat Sheet and journal — is introduced in the Week 1 workshop.

CW2 –

The CW2 project is briefed at the end of the Week 7 workshop, ahead of the Week 8 start. Each subsequent week's project stage (authentication, storage, dashboard, deployment) is briefed at the end of that week's workshop, before the lab in which it is built.

Sample assignments

CW-1

Exemplar solutions for each week are made available to students — *e.g. released after that week's submission deadline]*, to help contextualise the expected standard at Threshold, Typical and Excellent tiers.

CW 2

Exemplar project stages are made available to illustrating the expected standard for each week's contribution to the continuous project.

Draft feedback opportunities

CW1

ormative feedback is continuous rather than tied to a single session: it is given every week during the lab, and further support is available through the weekly 1-hour online drop-in session, where students can raise questions on that week's teaching and lab work, or email questions in advance for anonymous general feedback.CW1

CW2

As with CW1, feedback is continuous — given weekly during the lab session — with the online drop-in session available each week for questions on the current stage of the project, and office hours available for individual or private discussion..

Additional support

As a Middlesex student you can access to Librarians, Academic Skills and Maths teachers as well as IT specialists available to help you and provide advice.

Disability & Dyslexia Support (DDS)

The Disability & Dyslexia Support (DDS) team works to support students with a suspected or diagnosed disability, learning difference, or long-term health condition. The team supports with Learning Support Plans (LSPs) with agreed course adjustments; accessible teaching and assessment arrangements; adapted learning materials; Library support, including extended loan periods; permissions to record

lectures; screenings and diagnostic assessments; advice and support with Disabled Students' Allowances (DSA) applications; liaison with academic staff (with your consent).

*Medical evidence is **not required** to access a Learning Support Plan.*

Progression & Study Support

Advice and support is available if circumstances are impacting your studies, including understanding results, academic standing, and University regulations; support with attendance or assessment difficulties advice on changing course or study mode; taking a break from studies or leaving the University; understanding the academic, financial, or visa implications of study changes; interpreting progression decisions and identifying next steps.

How to Access Support

Students can book appointments with Student Success & Support services using the self-service booking system.

General contact points:

- **Disability & Dyslexia Support:** disability@mdx.ac.uk
- **Student Advice & Inclusion:** studentadviceteam@mdx.ac.uk
- **Mental Health & Wellbeing:** wellbeing@mdx.ac.uk

Feedback on your assignments

You will receive helpful and informative feedback on all assessments to support your learning and development. Feedback may be provided in various forms, including written comments, tutorial feedback (individual or group), peer feedback, or other appropriate methods. Formative or draft assessments will receive feedback but no grade, and comments will highlight strengths as well as areas for improvement. All assignments should be submitted online unless stated otherwise in the assessment brief.

Feedback on summative assessments will normally be provided within 15 working days of the published submission date.

Results Confirmation

How is your assignment mark agreed?

The following diagram provides an overview of the marking process for your module assessment:

Step 1	You submit your assignment
Step 2	The first marker grades the work and provides feedback; this could be completed anonymously depending on the assessment type.
Step 3	A moderator or second marker reviews a sample of the work to ensure the consistency of the grades and feedback. A final mark for the work is agreed between the first marker and the moderator or second marker.
Step 4	A sample of work, from a selection of modules across the programme, is sent to the External Examiner to check that the grading and feedback is at the right level and in line with external subject benchmarks (this applies to levels 5,6 &7 only).

Step 5	Tour final grades are submitted to the Programme assessment board.

Provisional Grades: At the end of your first semester, you can see your module grades in the Grades and Progress tile within MyMDX. These grades are provisional and not yet confirmed by the Programme Assessment Board.

Final Grades and Progression: After your second semester, the Programme Assessment Board will confirm your grades. Then, your final module results, progression status, or finalist classification will be posted in the Grades and Progress tile within MyMDX.

Each assessment component will be marked in percentages. However, the overall module grade will be calculated using the University 20-point scale. To access the 20-point scale, visit the MyMDX Assessment FAQ page.

Anonymous Marking Assessment Policy

We use anonymous marking, developed in partnership with MDXSU, so assessors cannot see your personal details when marking. This helps ensure fair and consistent assessment, although some assessment types are exempt under the Anonymous Marking Policy.

Extenuating Circumstances

If circumstances beyond your control affect your ability to meet an assessment deadline or perform well, these may be considered extenuating circumstances (ECs). If you find yourself in this situation, there are academic tools available to you to help you manage your studies. Please visit the MyMDX Extenuating Circumstances and Late Submission page for more information.

Expectations of studying

It is expected that you engage with online and on-campus in-person learning and activities. You are expected to behave in a professional, supportive manner to your peers and teachers. You must come to sessions prepared and ready to contribute where appropriate. Please conduct your email communication with fellow students, tutors and all relevant staff in a polite manner.

Appendix 1: Marking Criteria:

CW1

Criterion	Weight	What is assessed
Python knowledge and application	35%	Correct use of the concepts taught in Weeks 1–7, including values and types, decisions and loops, functions, data structures, files, classes, pandas, and string handling
Working mini-projects	25%	Functional weekly .py projects that meet the selected Threshold, Typical, or Excellent standard
Problem solving and debugging	15%	Ability to interpret errors, correct broken code, validate results, and make sensible technical decisions
Code quality and good practice	15%	Readable naming, clear structure, appropriate functions and classes, useful docstrings, and consistent formatting
Learning evidence and portfolio completion	10%	Completed lab notebooks, journal pages, Cheat Sheet Boxes 1–7, and an orderly record of progress
Total	**100%**	

CW2

Evidence	Requirement
project/ folder	Final working application, source code, and full Git commit history

Evidence	Requirement
Journal	Five handwritten pages, photographed and stored in journal/week08/ through journal/week12/
Cheat Sheet	Boxes 8–12 completed, giving 12 boxes in total across CW1 and CW2
Final presentation evidence	A short demo video, a code walkthrough, and a technical report submitted through Moodle
Deployment	Public URL for the working application